

Construction

Inverse Inversions

```
#include

using namespace std;
typedef long long ll;
const int maxN = 1e6+1;

int N, idx, ans[maxN];
ll K;
deque DQ;

int main(){
    scanf("%d %lld", &N, &K);
    for(int i = 1; i <= N; i++)
        DQ.push_back(i);

    for(int k = N-1; k >= 0; k--){
        if(k <= K){
            ans[++idx] = DQ.back();
            DQ.pop_back();
            K -= k;
        } else {
            ans[++idx] = DQ.front();
            DQ.pop_front();
        }
    }

    for(int i = 1; i <= idx; i++)
        printf("%d%c", ans[i], (" \n")[i==idx]);
}
```

Monotone Subsequences

```
#include

using namespace std;

int N, K;

void solve(){
    scanf("%d %d", &N, &K);
    if(K*K < N){
        printf("IMPOSSIBLE\n");
        return;
    }

    vector ans;
    int l = 1, r = K;
    while(true){
        for(int i = r; i >= l; i--)
            ans.push_back(i);
```

```

        if(r == N) break;
        l = r+1;
        r = min(r+K, N);
    }

    for(int i = 0; i < N; i++)
        printf("%d%c", ans[i], (" \n")[i==N-1]);
}

int main(){
    int T;
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        solve();
    }
}

```

Chess Tournament

```

#include
#include
#include

using namespace std;
using namespace __gnu_pbds;
typedef pair pii;

int N, K;
vector added, ans;
tree, rb_tree_tag,
tree_order_statistics_node_update> T;

int main(){
    scanf("%d", &N);
    for(int i = 1, x; i <= N; i++){
        scanf("%d", &x);
        T.insert({x, i});
    }

    for(int i = N-1; i >= 0; i--){
        pii P = *T.find_by_order(i);
        T.erase(T.find_by_order(i));
        int degu = P.first;
        int u = P.second;

        added.clear();
        for(int j = 1; j <= degu; j++){
            if(i-j < 0 || (*T.find_by_order(i-j)).first == 0){
                printf("IMPOSSIBLE\n");
                return 0;
            }

            pii Q = *T.find_by_order(i-j);
            T.erase(T.find_by_order(i-j));
            int degv = Q.first;
            int v = Q.second;

            ans.push_back({min(u, v), max(u, v)});
            added.push_back({degv-1, v});
        }
    }
}

```

```
        for(pii Q : added)
            T.insert({Q.first, Q.second});
    }

    sort(ans.begin(), ans.end());
    K = (int) ans.size();
    printf("%d\n", K);
    for(int i = 0; i < K; i++)
        printf("%d %d\n", ans[i].first, ans[i].second);
}
```
